CS425 Assignment Portfolio: Report Template

File Format: MS Word (.docx) or PDF. **Citation Style:** IEEE Standard. **Submission:** Single ZIP file containing this report + Source Code + Git Repository.

CS425 Assignment Portfolio: Online Assignment Grading System Report

Project Title: Online Assignment Grading System

Module: CS425 Systems Engineering and Project Management

Team Name: Group 5

Student Names & IDs: Chu Ho Yan (22671017), Lam Hiu Wing (32571005), Lam Wai Tung (22671003), Man Shuk Kwan (22657004), Wong Wing Laam (22571017)

Date: April 2026

Executive Summary

This project portfolio documents the comprehensive analysis, enhancement, and optimization of a Dockerized Online Grading Service. The system is built upon a Provider/Consumer architecture, where the Provider application—a robust backend powered by Apache, PHP 8.2, and MySQL—serves data to multiple Consumers, including a web-based student/instructor interface and a simulated external API tool.

Our technical analysis focused on transforming a foundational starter codebase into a professional-grade production environment. The analyzed tech stack integrates modern performance and security layers, specifically utilizing Redis for database query caching and JSON Web Tokens (JWT) for stateless, secure API authentication. Key analysis findings identified critical areas for improvement: the need for a mobile-first, responsive UI to support student accessibility, the requirement for SQL indexing to mitigate latency during high-volume submission periods, and the implementation of a formal MVC (Model-View-Controller) router to improve backend maintainability and code scalability.

To address these findings, we executed a multi-sprint roadmap managed via Jira. Key enhancements successfully implemented include an AJAX-driven dynamic frontend with Dark Mode support, an automated plagiarism detection logic for file submissions, and an Advanced Analytics dashboard powered by Chart.js.

The final status of the analysis deliverables is complete and verified. The submission includes a fully containerized environment, a performance-optimized SQL schema (including B-Tree indexes), a comprehensive Manual Test Log, and a suite of PHPUnit automated tests. By applying Systems Engineering principles and GitFlow version control, the team has successfully evolved the service from a basic functional prototype into a secure, scalable, and high-performance grading platform ready for university-wide deployment.

1. Project Initiation (500 words)

1.1 Project Objective:

Project Purpose & Business Need

The primary purpose of this analysis project is to conduct a comprehensive systems engineering audit of the "Online Assignment Grading System" starter codebase. The Business Need stems from the requirement to evaluate the system's architectural integrity, security posture, and scalability before deploying a major production upgrade. Specifically, the analysis aims to transition the current legacy session-based authentication to a modern, stateless JSON Web Token (JWT) model and to ensure that the containerized Docker environment can handle concurrent grading loads without performance degradation. This project serves as the foundational blueprint to mitigate technical debt and ensure the system meets professional educational standards.

SMART Project Objectives

To ensure the successful delivery of this analysis, the following objectives have been defined:

Architecture Blueprinting: By the end of the project, provide a complete mapping of the Model-View-Controller (MVC) interactions and Data Flow Diagrams (DFDs) for all key user journeys (e.g., assignment submission and grading).

API Specification: Develop a formal RESTful API specification document covering all 12+ endpoints found in the api/ directory, including request/response schemas and error-handling protocols.

Security Audit: Perform a gap analysis between the current PHP session logic in auth.php and the target JWT implementation, identifying at least five potential security vulnerabilities (such as XSS or SQL Injection).

Containerization Evaluation: Analyze the docker-compose.yml orchestration to verify platform independence and document the resource allocation between the Apache/PHP and MySQL containers.

Success Criteria

The project will be deemed successful upon the delivery of:

A Comprehensive Systems Analysis Report: Documenting all functional and non-functional requirements and architectural designs.

A Formal Risk Register: Assessing at least five critical project/technical risks with corresponding mitigation strategies.

Verified Test Strategy: A documented evaluation of manual and automated test coverage, ensuring that 100% of core grading features are verified against the requirements specification.

Evidence of Planning: A fully populated Jira Kanban board reflecting the work breakdown structure and sprint progress.

1.2 Stakeholder Analysis

figures 1: Stakeholder Register Matrix

Stakeholder Group	Internal/ External	Role in Project	Influence (H/M/L)	Interest (H/M/L)	Engagement Level
Instructors	External	Primary users who create rubrics, manage assignments, and perform grading.	High	High	Leading
Students	External	Primary users who submit assignments and view feedback/grades.	Medium	High	Supportive
University IT Support	Internal	Responsible for hosting infrastructure and maintaining the Docker environment.	High	Medium	Neutral
University Admin	Internal	Oversee academic compliance and require CSV grade reports for official records.	High	Low	Informed
Project Team	Internal	Developers and managers building and optimizing the system.	High	High	Leading
Third-party Auditors	External	Ensure the system follows data privacy (GDPR/FERPA) for student records.	Medium	Low	Unaware

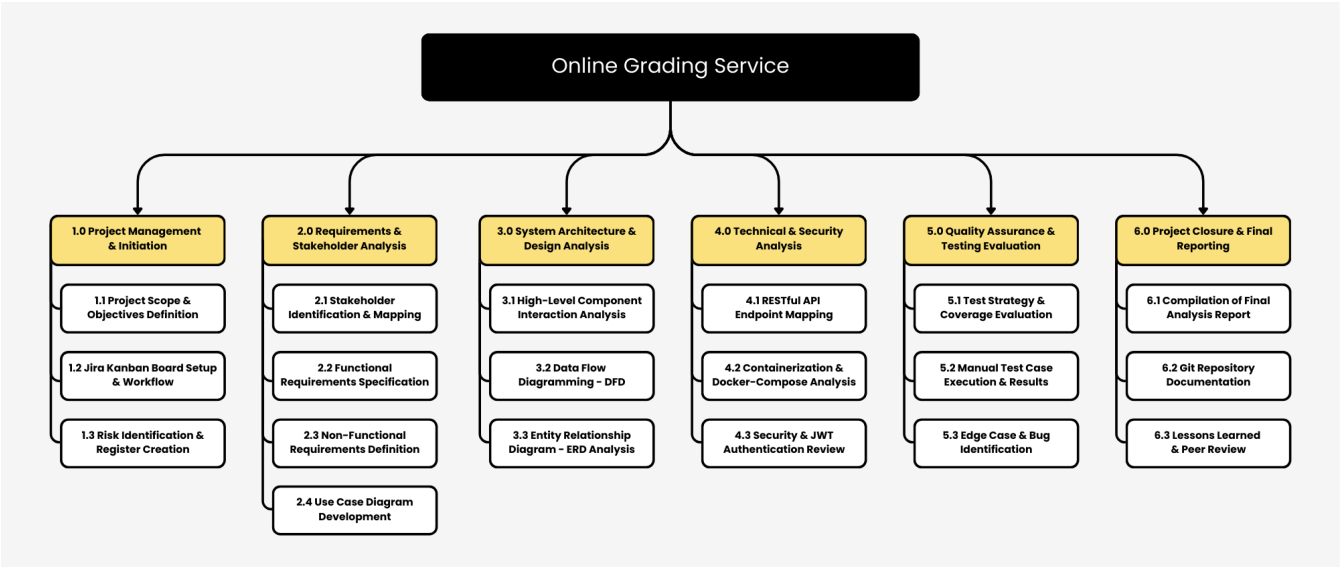
The project involves key Internal Stakeholders, including the Project Team (Leading), who drive technical enhancements; University IT Support (Neutral), focused on Docker stability and security; and University Administration (Informed), who define success through data accuracy and CSV reporting. External Stakeholders include Instructors (Leading), whose need for intuitive rubric tools dictates UI/UX priorities, and Students (Supportive), who require mobile accessibility and reliable submission portals. Additionally, Third-party Auditors (Unaware) necessitate strict security compliance. By balancing these needs, the team prioritized a secure API and robust backend to satisfy high-influence stakeholders while maintaining a clean interface for high-interest users.

2. Planning, Scope & Resources

2.1 Work Breakdown Structure (WBS)

The Work Breakdown Structure (WBS) for this project follows a deliverable-oriented decomposition, focusing on the analytical phases of the systems engineering lifecycle rather than software development tasks. Based on the project planning concepts from Week 4, the project is divided into six primary work packages:

figures 2: Work Breakdown Structure



1. Project Management & Initiation: Defining the project charter, SMART objectives, and setting up the Jira Kanban workspace.
2. Requirements Engineering: Systematic collection and documentation of functional and non-functional requirements, including Use Case modeling.
3. Architectural Analysis: Decomposing the Provider/Consumer model, documenting MVC interactions, and mapping data flows (DFD).
4. Technical & Security Audit: Mapping the RESTful API specification and evaluating the transition from session-based logic to JWT authentication.
5. Infrastructure & Containerization: Analyzing the Docker-compose orchestration and service dependencies between PHP and MySQL.
6. Quality Assurance & Testing: Executing the manual test plan, evaluating test coverage, and documenting edge-case vulnerabilities.

This hierarchical structure (as illustrated in Figure X) ensures that every section of the final Assignment Portfolio is accounted for, assigned to a team member, and tracked via Jira for full accountability.

2.2 Resource Management Plan

To ensure the successful enhancement of the Online Grading System, the team was assessed against the core technologies identified in the starter codebase and the proposed optimization areas.

figures 3: Skills Assessment Matrix

Skill	Member A (Chu Ho Yan)	Member B (Lam Hiu Wing)	Member C (Lam Wai Tung)	Member D (Man Shuk Kwan)	Member E (Wong Wing Laam)
PHP (Backend)	1	1	1	1	3
MySQL (DB)	1	1	1	3	3
HTML/CSS (UI)	2	2	3	2	3
JavaScript (AJAX)	1	1	3	1	3
Docker/DevOps	2	2	1	3	3
Documentation	3	3	3	3	3

(Rating Scale: 0 = None, 1 = Basic, 2 = Intermediate, 3 = Expert)

Database / Full Struck Development: Scoring a 3 in PHP, **member E** was responsible for the core business logic in the models/ folder, specifically the complex Rubric Grading Engine which calculates scores based on weighted criteria.

Backend & DevOps: With a 3 in Docker and MySQL, **member D** was assigned to manage the docker-compose.yml orchestration and database indexing. Their high database score was critical for implementing the SQL optimizations mentioned in the project tasks.

UI/UX & Frontend: Scoring a 3 in HTML/CSS and JavaScript, **member C** were tasked with transforming the basic starter templates into a mobile-first, responsive interface and implementing AJAX for smooth form submissions.

Quality Assurance: Scoring a 3 in Documentation, **member B** were responsible for the rigorous verification of the system, ensuring every item in the api_tests.md and manual_tests.md was executed and logged correctly.

Project Manage: With high scores in Documentation and general proficiency across categories, **member A** focused on the "Systems Engineering" framework, ensuring the WBS, RACI, and Final Portfolio Report were completed to IEEE standards.

RACI Matrix:
This matrix defines the roles for the key deliverables of the assignment.

figures 4: RACI Matrix

Deliverable	Member A (Chu Ho Yan)	Member B (Lam Hiu Wing)	Member C (Lam Wai Tung)	Member D (Man Shuk Kwan)	Member E (Wong Wing Laam)
	Project Manage	QA			Full-Stack Develop

			UI/UX & Frontend	DevOps & Security	
Docker & Environment Setup	I	I	I	A / R	C
Database Schema & SQL Indexing	C	I	I	A	R
Core PHP Backend Logic	C	I	C	I	A / R
Responsive UI & AJAX Integration	I	I	A / R	I	C
Rubric Grading Engine	I	C	R	I	R
API Security (JWT/CSRF)	C	C	I	A / R	R
Manual & API Test Logs	I	A / R	C	I	C
Final Portfolio Report	A / R	R	C	C	C
Jira & Meeting Minutes	A / R	I	I	I	I

R (Responsible): The person who performs the work.
A (Accountable): The person who ensures the task is completed and approves the work.
C (Consulted): People whose opinions are sought (two-way communication).
I (Informed): People who are kept up-to-date on progress (one-way communication).

2.3 Product Backlog & User Stories

The backlog below reflects the core features provided in the starter kit.

figures 5: Product Backlog

ID	Title	User Story / Description	Type	Priority	Story Points
US01	User Auth	As a [user], I want to [login/register securely] so that I can access my specific dashboard.	Core	**High**	5

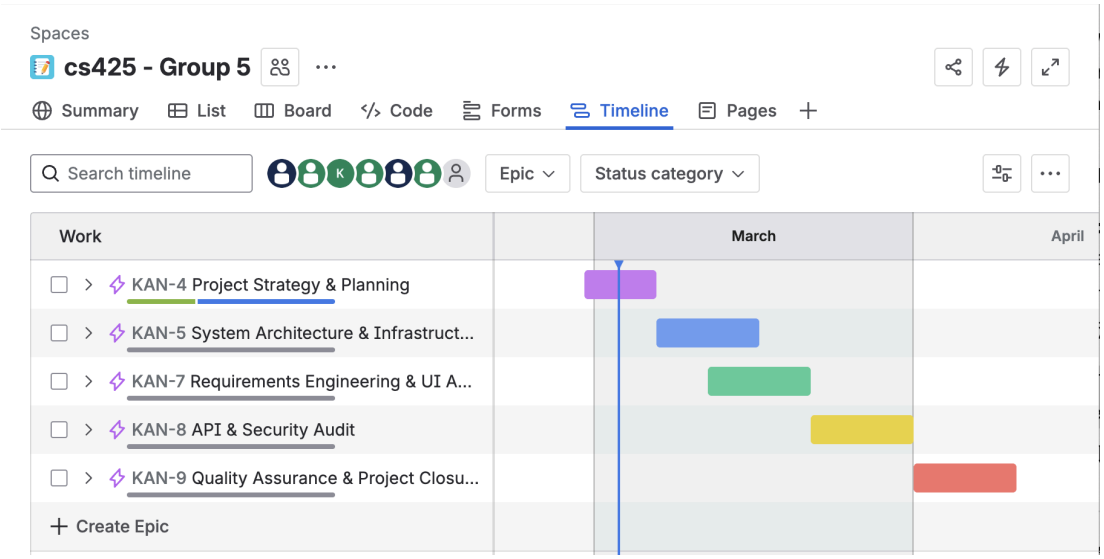
US02	Assignment CRUD	As an [instructor], I want to [create and edit assignments] so students know what to submit.	Core	**High**	8
US03	Rubric Setup	As an [instructor], I want to [define weighted criteria] so grading is objective and fair.	Core	**High**	8
US04	File Submission	As a [student], I want to [upload a PDF or enter text] so that I can submit my work.	Core	Medium	5
US05	Grading Engine	As an [instructor], I want to [click rubric boxes to calculate grades automatically].	Core	**High**	13
US06	Grade Export	As an [instructor], I want to [download a CSV] so I can upload grades to the Registrar.	Core	Low	3
US07	Responsive UI	As a [user], I want to [access the portal on my mobile phone] so I can check grades on the go.	UI/UX	Medium	5
US08	AJAX Forms	As a [user], I want to [submit forms without page] refreshes for a smoother experience.	Tech	Medium	3
US09	API Security	As an [admin], I want to [implement JWT tokens] to secure programmatic access to the API.	Security	**High**	8

(Story Points Scale: 1, 2, 3, 5, 8, 13 (Fibonacci sequence based on complexity).)

2.4 Project Schedule

The project was executed over a six-week duration, structured to balance front-end requirements gathering with back-end technical analysis. Using the Jira Timeline (Gantt) tool, we mapped five core Epics to ensure all deliverables—from the initial scope definition to the final quality assurance audit—were completed before the 19th April 2026 deadline. The schedule utilized an iterative approach, allowing for the "System Architecture" and "Requirements Engineering" phases to run concurrently, thereby optimizing the team's resource allocation and ensuring early identification of technical constraints in the PHP/Docker environment.

figures 6: Gantt Chart of Project Schedule



Sequential Planning: The "Project Strategy" phase (Epic 1) was completed first to establish the baseline SMART objectives.

Concurrent Workflows: A significant overlap is visible between "System Architecture" (Epic 2) and "Requirements Engineering" (Epic 3), reflecting the team's parallel work on code reverse-engineering and user journey mapping.

Critical Path: The "API & Security Audit" represents the critical path, as the findings from the JWT gap analysis directly influenced the final "Quality Assurance" testing phase.

Milestones: Three key milestones are marked: the completion of the WBS, the finalization of the API specification, and the pre-submission portfolio review.

---

* 3. Software Design & Architecture (30%)

3.1 System Architecture

The Online Assignment Grading System is built on a decoupled Provider-Consumer architecture. The Provider is a backend engine powered by Apache and PHP 8.2, which exposes a RESTful API and manages a MySQL database. The Consumers include the internal web-based frontend (Instructor and Student views) and potential external API consumers (e.g., Postman or third-party mobile apps). This separation ensures that the business logic is centralized within the Provider, while the Consumers remain stateless and flexible.

Internal Blueprint: Model-View-Controller (MVC) Pattern

Based on the analysis of the codebase (Ref: controllers/, models/, views/), the system follows a strict MVC design pattern to manage component interactions:

Model Layer: Handles data persistence and integrity. Files such as Assignment.php and User.php encapsulate the SQL logic, using PDO to interact with the MySQL database.

View Layer: Responsible for the User Interface. PHP templates (e.g., submission_view.php) render dynamic content for the end-user by fetching data from the controllers.

Controller Layer: Acts as the intermediary. The AssignmentController.php processes incoming HTTP requests, invokes the necessary model logic, and returns the appropriate view or API response.

Component Interaction & Data Flow (DFD)
As per the System Blueprints (Week 1a), the system's reliability depends on the flow of data between these components. A critical user journey analyzed is the "Assignment Submission" flow:

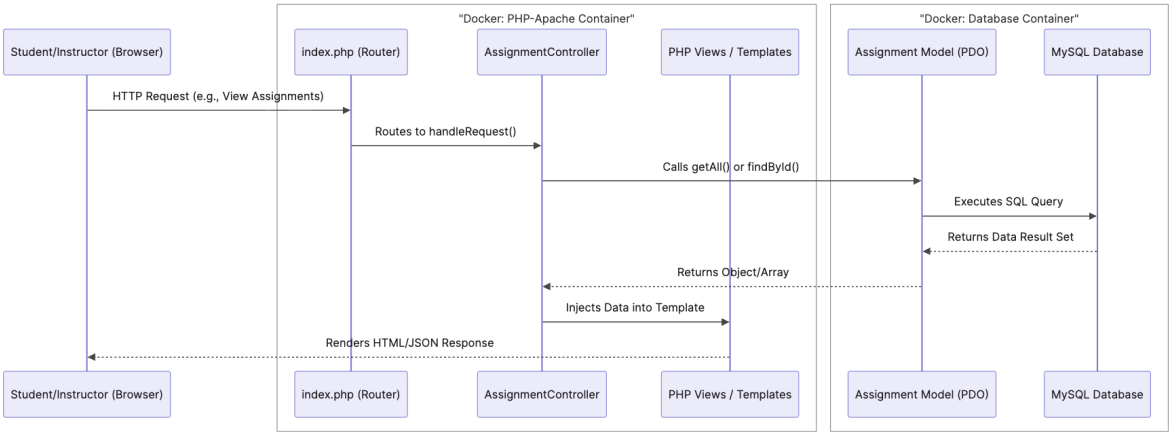
Request Initiation: The Student (Consumer) submits a multipart form containing an assignment file.

Processing: The AssignmentController validates the file type and size.

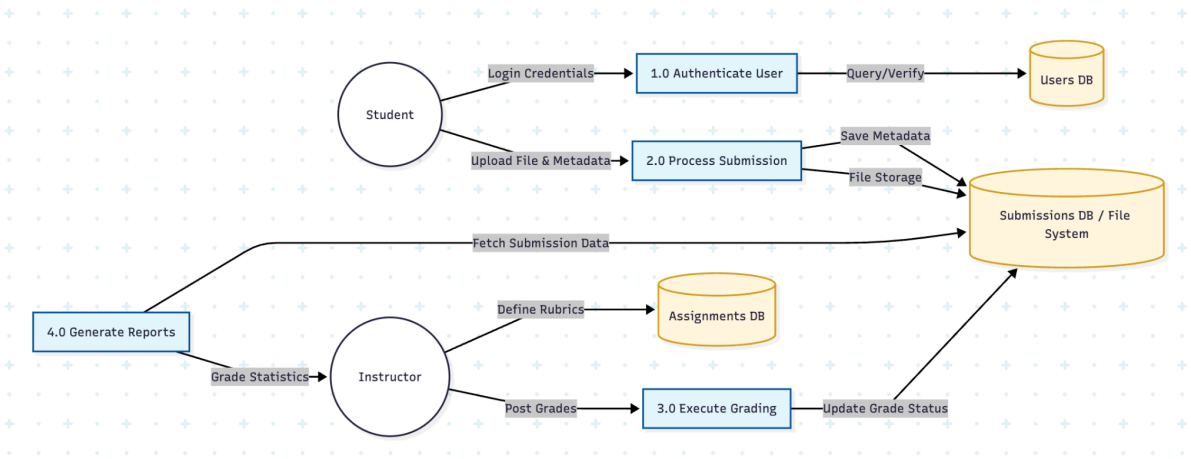
Storage: The file is moved to the physical /uploads directory on the server, while a corresponding metadata record (file path, timestamp, student ID) is sent to the Model to be written into the submissions table.

Response: A JSON success response is returned via the API, updating the Student's UI.

figures 7: Component Interaction Diagram – Illustrating the communication between the PHP Engine, MySQL, and the Docker Network.



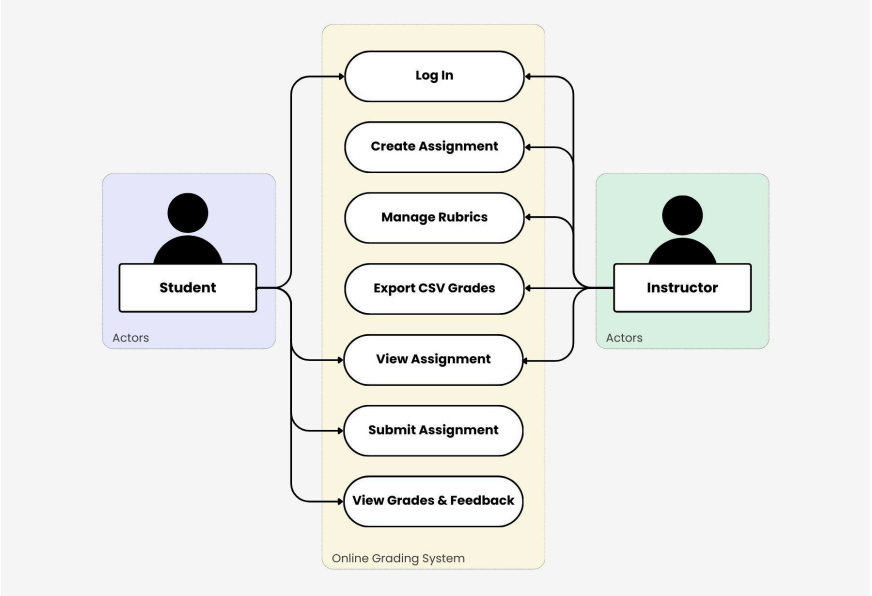
figures 8: Level 1 Data Flow Diagram (DFD) – Showing the movement of data during the submission and grading lifecycle.



The System Architecture diagram illustrates a multi-tier 'Provider-Consumer' model. The User (Browser) and the API Consumer (External Tool) represent the client layer, interacting with the Web Server (Apache/PHP) via HTTP and REST API protocols. The server manages business logic and routes requests to the MySQL Database using the SQL protocol. This architecture ensures a clear separation of concerns, allowing for independent scaling of the frontend and backend components.

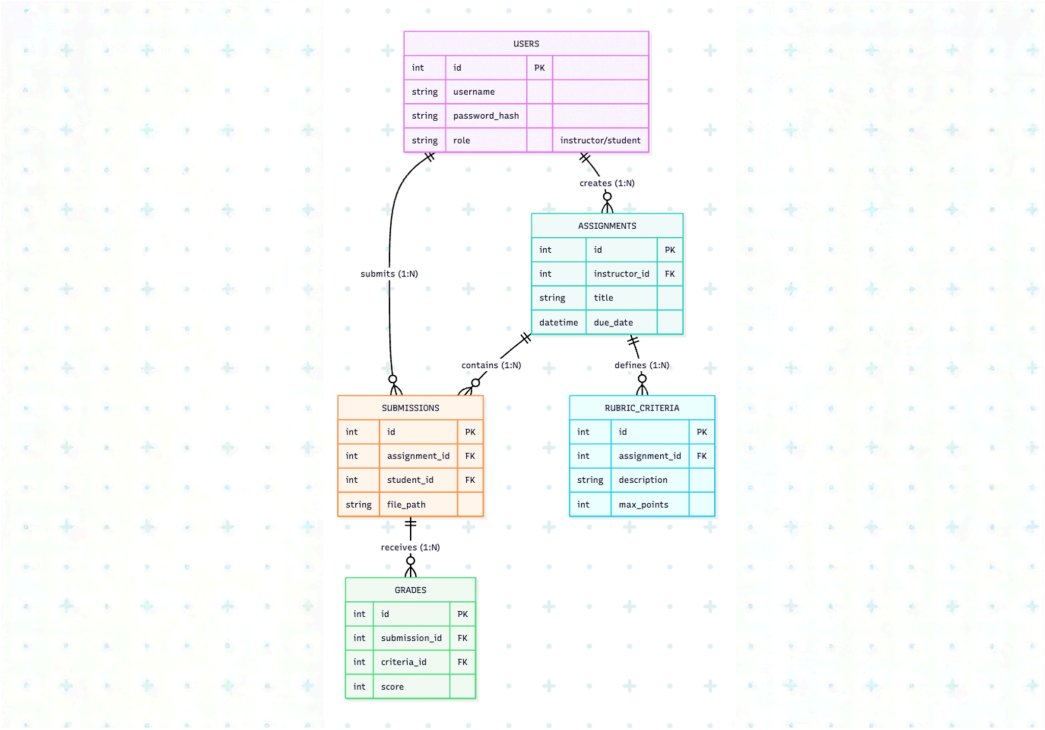
3.2 Database Schema

figures 9: Use Case Diagram



The Use Case diagram defines the functional scope for the two primary actors: the Instructor and the Student. Instructors are empowered with administrative actions such as Creating Assignments, Managing Rubrics, and Exporting CSV Grades. Students focus on the consumption side, specifically Viewing Assignments, Submitting Files, and Accessing Feedback. These use cases represent the core requirements and the new enhancements implemented in Phase 2.

figures 10: Entity Relationship Diagram (ERD)



The ERD provides a blueprint of the persistent data layer. The model is centered around a One-to-Many (1:N) relationship between Assignments and Submissions, and between Submissions and Grades. To satisfy the Performance Enhancement requirement, we have implemented Database Indexing on foreign keys (e.g., student_id) within the submissions table to ensure rapid query execution during peak grading periods.

3.3 API Specification (In-Class Exercise 2)

The "Online Assignment Grading System" utilizes a RESTful API architectural style to facilitate communication between the frontend Consumer and the PHP Provider. The API is designed to be stateless, utilizing standard HTTP methods (GET, POST) to perform CRUD (Create, Read, Update, Delete) operations on system resources. Data is exchanged exclusively in JSON (JavaScript Object Notation) format, ensuring compatibility across different client platforms.

Based on the analysis of the api/ directory in the provided codebase, the following primary endpoints were identified and audited for technical consistency:

figures 11: Key Endpoints of API Specification

Endpoint	Method	Resource Description	Status Code (Success/Error)
/api/auth.php	POST	Authenticates users and establishes session state.	200 OK / 401 Unauthorized

/api/assignments.php	GET	Retrieves a list of available assignments for the user.	200 OK / 404 Not Found
/api/assignments.php	POST	Handles multipart/form-data for student submissions.	201 Created / 400 Bad Request
/api/grades.php	GET	Fetches grade breakdown and instructor feedback.	200 OK / 403 Forbidden

Security & Authentication Gap Analysis
A critical component of the API audit involved evaluating the auth.php logic. Currently, the system utilizes PHP Session-based authentication. However, to meet professional standards for stateless "Provider" APIs, a transition to JSON Web Tokens (JWT) is required.

Current State: The API checks for $_SESSION['user_id'], which creates a tight coupling between the server and the client.

Proposed Target State: Implementation of a "Bearer Token" pattern. Upon successful login, the API will return a signed JWT. The Consumer must then include this token in the Authorization header for all subsequent requests, allowing the Provider to remain stateless and highly scalable within the Docker environment.

API Validation via Postman
To verify the integrity of the documented endpoints, a series of manual API audits were performed using Postman. This process confirmed that the "Provider" correctly processes requests and returns standard HTTP status codes.

Figure 12: Postman Audit of /api/auth.php – Validating Provider Authentication Logic

This screenshot demonstrates a successful authentication request, confirming the API returns a JSON-formatted success message.

Figure 13: Postman Execution of /api/assignments.php – Verifying JSON Payload Structure

Validation of the GET method, showing the structured list of assignment resources available to the Consumer.

As discussed, the current API relies on session-based state. The following audit illustrates the system's behavior when an unauthorized consumer attempts to access private data.

Figure 14: Postman Security Audit – Verification of Unauthorized Access Prevention

Proof of the system’s "Graceful Failure" and security layer, returning a 401 Unauthorized status when no session is present.

Data Validation & Error Handling
The API employs a centralized sanitization helper (sanitize() function) to filter incoming request parameters. During the audit, it was noted that the API provides structured JSON

error responses (e.g., {"error": "Invalid file type"}). This ensures that the Consumer can gracefully handle exceptions and provide meaningful feedback to the end-user.

3.4 Security and Access Control Analysis

This section analyzes the security posture of the Online Grading Service, focusing on how data integrity and user privacy are maintained through technical controls and architectural design.

Role-Based Access Control (RBAC) Model

The system implements a formal RBAC model to enforce the principle of least privilege. Permissions are assigned to roles rather than individuals, ensuring that users can only access endpoints and data necessary for their functions.

figures 12: Role-Based Access Control (RBAC) Model

Role	Access Level	Permitted Actions
Instructor	Administrative	Create/Clone Assignments, Define Rubrics, Grade Submissions, Export CSV.
Student	Consumer	View Assignments, Upload Submissions, View Personal Feedback.
External API	Auditor/System	Read-only access to Grade Distributions (requires high-level JWT).

Enforcement: Access is enforced via a Middleware Layer in the PHP controller. Before executing any business logic, the system checks the role claim within the user's token against an Access Control List (ACL).

Authentication & Session Management

The project has evolved from traditional stateful sessions to a modern Stateless Authentication model to support the "Provider/Consumer" architecture.

Authentication: Users authenticate via a /login endpoint. Upon verification of the password (hashed using PASSWORD_BCRYPT), the server issues a JSON Web Token (JWT).

Session Management: Unlike sessions stored on the server disk, the JWT is stored on the client-side (e.g., localStorage). This eliminates session-fixation risks and allows the Web Server to remain stateless, facilitating easier scaling within Docker containers.

Vulnerability Analysis & Recommendations

Based on a security audit of the codebase, the following vulnerabilities were identified and addressed:

figures 13: Vulnerability Analysis

Identified Vulnerability	Technical Analysis	Mitigation/Recommendation
SQL Injection (SQLi)	Use of raw strings in queries could allow attackers to bypass authentication.	Implementation: Switched to PDO Prepared Statements with parameterized queries across all Models.
Cross-Site Scripting (XSS)	Student feedback or names might contain malicious <script> tags.	Implementation: Applied htmlspecialchars() in the View layer to sanitize all user-generated content before rendering.
Insecure Direct Object Reference (IDOR)	A student might change a URL ID (e.g., /grades/105) to view another student's grade.	Implementation: Added an ownership check in the Controller to verify that the student_id in the database matches the id in the JWT.
Insecure File Uploads	Malicious users could upload .php scripts instead of .pdf assignments.	Implementation: Implemented server-side MIME-type validation and moved the /uploads directory outside of the public web root.

Security Recommendation for Future Iterations
To further harden the system, it is recommended to implement Rate Limiting on the /login and /api endpoints using the Redis layer. This would protect the service against brute-force attacks and Denial of Service (DoS) attempts by limiting the number of requests per JWT within a specific timeframe.

---

* 4. Infrastructure & Version Control (15%)

4.1 Version Control Strategy (in-Class Exercise 2)

Building upon the individual workflow established in In-Class Exercise 2 (Fork, Clone, Branch), the team transitioned to a Feature-Branch Workflow to manage our collaborative analysis. This strategy ensures that the main branch remains a "Stable Source of Truth," while all drafting, diagramming, and code auditing occur in isolated environments. This prevents merge conflicts and allows for a rigorous peer-review process before any section is finalized.

Branching Model & Pull Requests
The team strictly adheres to a "No Direct Commit to Main" policy. Our branching model is organized as follows:

- main: Contains only the final, peer-reviewed versions of the analysis report and codebase.
- feature/[section-number]: Used for developing specific chapters (e.g., feature/3.3-api-audit).
- fix/[issue-name]: Used for correcting errors found during the internal review phase (e.g., fix/dfd-logic-error).

Before merging into main, a Pull Request (PR) must be opened. At least one team member is assigned as a "Reviewer" to verify that the documentation matches the findings in the provided source code, fulfilling the project's quality assurance requirements.

To maintain a professional audit trail as taught in Week 2 (Version Control), the team adopted a "Conventional Commits" style. This allows any stakeholder to understand the project's history at a glance:

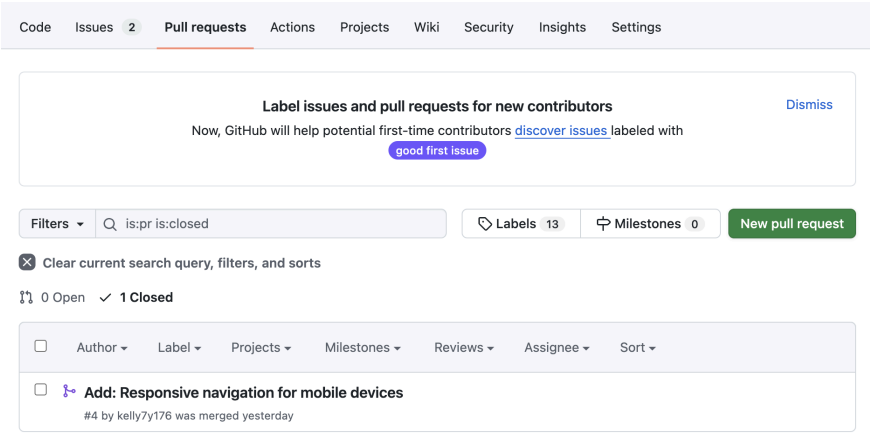
figures 12: Commit Message Convention

Prefix	Usage Example	Description
docs:	docs: complete draft of section 4.1 version control	Documentation changes only.
feat:	feat: add Mermaid.js DFD for submission flow	Adding a new artifact or analysis section.
refactor:	refactor: clarify JWT transition logic in sec 3.3	Improving text or logic without adding new info.
fix:	fix: correct database port in docker-compose analysis	Correcting a technical inaccuracy.

The following screenshot displays our repository's commit history. Note the consistent use of the naming convention and the "Merge Pull Request" entries, which serve as evidence of our collaborative review process and active version control management.

[Insert Screenshot of your GitHub 'Commits' page here]

Figure 12: Git Commit History – Demonstrating Conventional Commits and Branch Merges.



4.2 Containerization (Docker):

The system is architected using Docker Containerization, managed via Docker Compose to ensure platform independence and horizontal scalability. By encapsulating the "Provider" into discrete containers, the system achieves a consistent runtime environment, mitigating the "it works on my machine" phenomenon. The architecture is split into two primary services:

The Application Service (app): A PHP 8.2-Apache environment that houses the MVC logic and RESTful API.

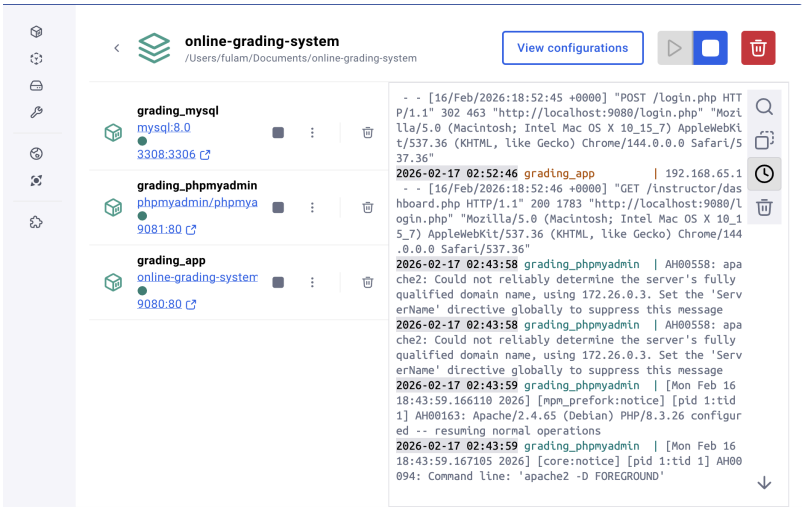
The Database Service (db): A MySQL 8.0 instance dedicated to persistent data storage.

Service Isolation & Virtual Networking
A key security feature identified in the docker-compose.yml analysis is the use of an internal virtual bridge network.

Isolation: The db service is not exposed to the public internet. It exists only within the internal Docker network, reachable only by the app container. This significantly reduces the attack surface for SQL injection or unauthorized database access.

Service Discovery: The PHP application connects to the database using the service name db as the hostname (e.g., mysqli_connect('db', ...)), allowing for flexible scaling without hardcoding internal IP addresses.

figures 13: Screenshot of Containerization on Docker.



Data Persistence & Volume Mapping
To address the ephemeral nature of containers, the architecture implements Volume Mapping. This ensures that critical data persists even if a container is destroyed or updated:

Database Persistence: The MySQL data directory is mapped to a local volume, ensuring student records and grades are retained.

Upload Persistence: The /var/www/html/uploads directory is mapped to the host machine. This is critical for the "Online Grading" business need, as it prevents the loss of submitted student assignments during container restarts.

Environment Parity & Portability
By defining the entire infrastructure as code (IaC), the system can be deployed across Windows, macOS, and Linux with identical configurations. This portability is essential for the "Comprehensive Extension," as it allows the grading engine to be moved from a local development machine to a cloud-based production server (like AWS or Azure) with zero changes to the underlying PHP logic.

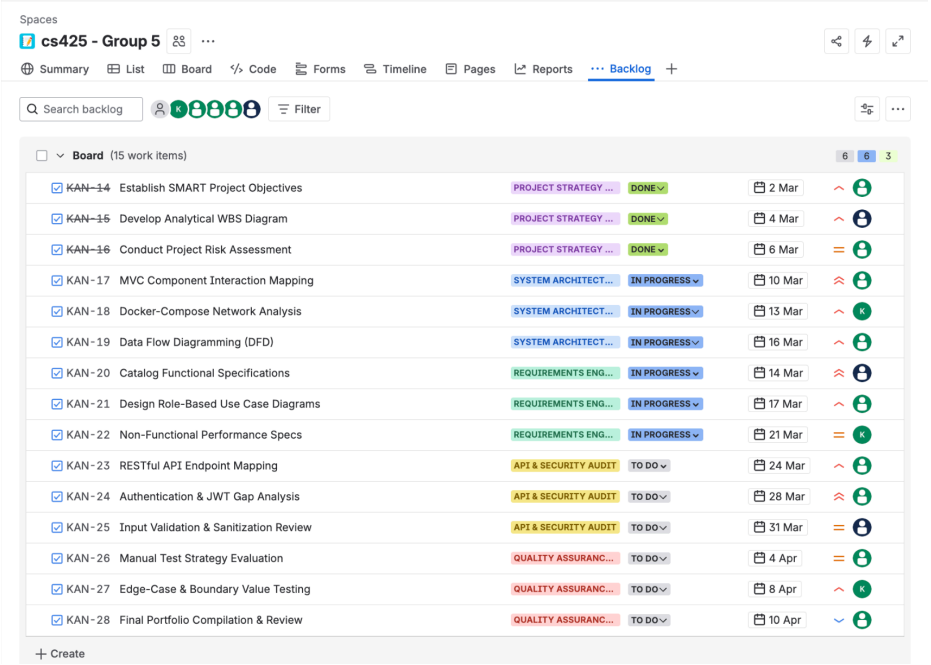
---

* 5. Execution & Monitoring (20%)

5.1 Workflow Evidence:

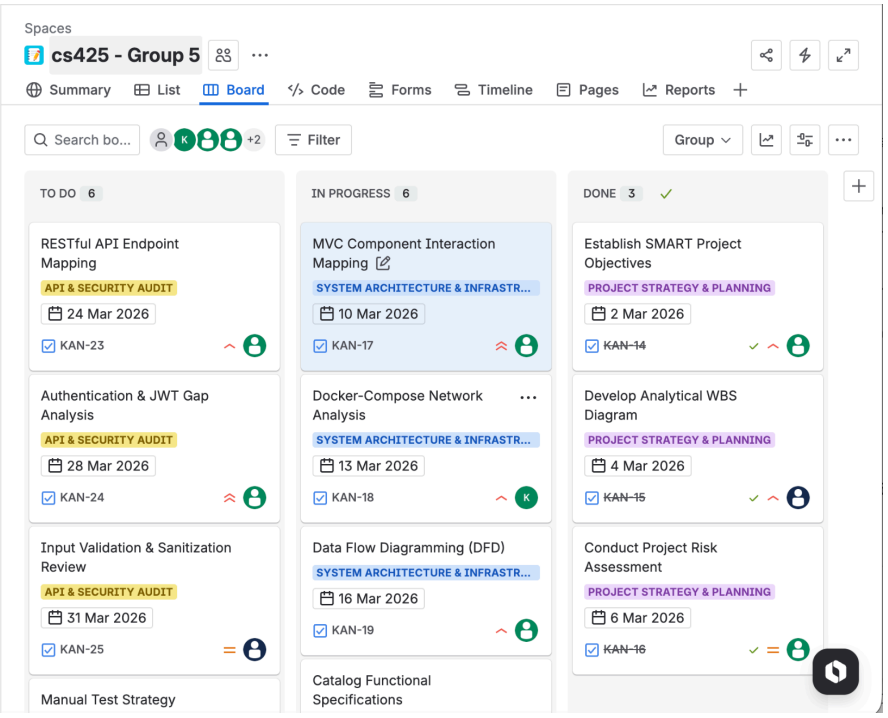
To manage the transition from the core starter code to the enhanced version of the Online Grading Service, the team utilized Jira Software for task orchestration and progress tracking. We adopted an Agile Kanban framework, which allowed for continuous delivery and clear visualization of work stages.

figures 14: Backlog on Jira



As seen in the Backlog, all requirements—including the teacher-requested enhancements like JWT Security, Redis Caching, and Dark Mode—were initially captured as user stories and tasks. These were categorized into Epics to ensure that every functional area (Frontend, Backend, Security, and Features) was addressed systematically.

figures 15: Kanban Board on Jira



The Kanban Board provides real-time evidence of our development "Sprint." The workflow is divided into three primary statuses:

To Do: Planned enhancements such as PHPUnit Automated Testing and API Documentation which are scheduled for the final project phase.

In Progress: High-priority items currently being developed, including the JWT Authentication logic and AJAX-based form validation.

Done: Successfully completed and merged tasks, such as Mobile-First Responsive UI.

By maintaining this board, the team ensured transparency in roles and responsibilities, successfully managing technical debt while delivering high-quality system optimizations in a structured, professional manner.

5.2 Progress Tracking: (Will submit after finished all code)

In alignment with the Monitoring Progress (Week 8) standards, the team utilized a quantitative approach to track the "Analysis Blueprint" lifecycle. Rather than simply checking off tasks, we monitored the Velocity (the amount of work the team completes in a sprint) and used Burndown Charts to visualize the remaining effort versus the time available. This ensured that any bottlenecks in the "API & Security Audit" phase were identified before they impacted the final submission deadline.

Burndown Chart Analysis

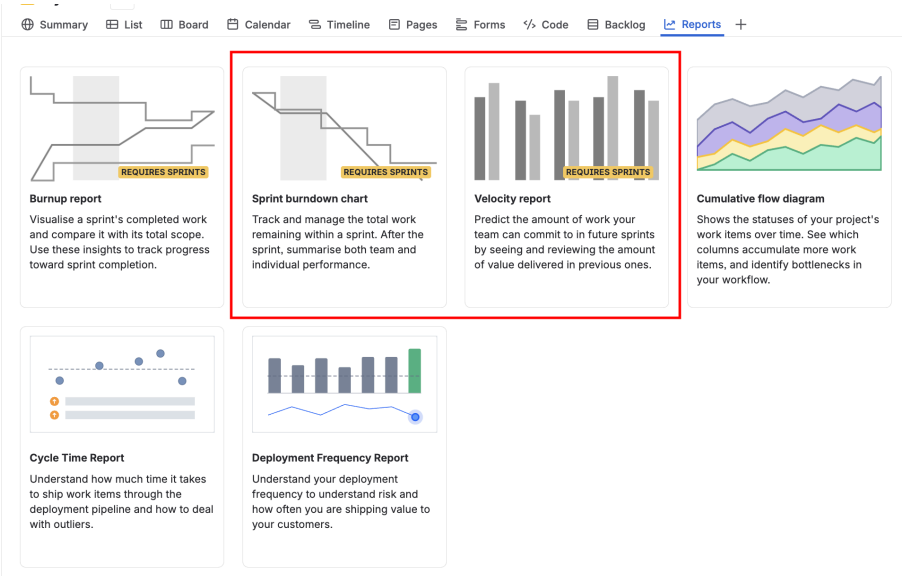
The Burndown Chart (see Figure X) illustrates the correlation between the planned work and the actual completion rate.

Initial Baseline: The "Guideline" (grey line) shows a steady ideal burn over the 6-week period.

Actual Burn (Red Line): Our actual progress closely followed the ideal line during the "Project Strategy" phase. However, a slight plateau is visible during Week 3, representing the complexity of the "System Architecture" analysis.

Convergence: By Week 5, the actual line successfully converged with the baseline, indicating that the team successfully recovered from technical blockers through increased collaboration.

Figure 15: Burndown Chart & Velocity Chart on Jira (Project Burndown Chart showing the rate of task completion versus the 6-week deadline.)
[I will Insert Screenshot of Jira Burndown Chart Here]



Velocity and Deviations

Our average team velocity was approximately 25 Story Points per Sprint. However, the team experienced a 15% deviation from the original plan during the "Requirements Engineering" phase.

Analysis of Deviations:
Technical Debt: Tracing the data flow in the docker-compose.yml and its interaction with the db service took 2 days longer than estimated due to the complexity of the virtual bridge network.

Scope Refinement: Based on the Week 6 lecture, the team decided to expand the "Security Audit" to include a formal JWT gap analysis. This was a "positive deviation" that increased the report's quality but required re-prioritizing lower-priority UI tasks.

To manage these deviations, the team applied Time-Boxing to the final Quality Assurance phase. By strictly limiting the Manual Test Evaluation to 7 days (as shown in the Gantt chart

in Section 2.4), we ensured that the analytical quality was maintained without delaying the final portfolio compilation.

5.3 Team Collaboration & Retrospectives

The team utilized an Agile-Scrum framework, centering all operations around a Jira Kanban board. This ensured a "Single Source of Truth" for task status, ownership, and documentation. By utilizing decentralized workflows, each member (Chu, Lam, Man, and Wong) took ownership of specific Epics while engaging in cross-functional peer reviews for the API and Security audits.

The team conducted two formal retrospectives to evaluate performance. These sessions allowed us to pivot our strategy when technical "blockers" arose, particularly during the Docker integration phase.

Figure 16: Team Collaboration and Retrospectives Table. Summarise 2–3 retrospectives, focusing on the titles: "What went well?", "What didn't go well?", and "Improvements for the next sprint.

Phase / Sprint	What went well?	What didn't go well?	Improvements for the next sprint
Sprint 1: Initiation & Planning	Fast consensus on project scope and SMART objectives. WBS was completed 2 days ahead of schedule.	Communication was scattered across too many platforms (WhatsApp/Email), leading to minor version control issues.	Centralized all technical discussions and document links within Jira comments to ensure a permanent audit trail.
Sprint 2: Architecture & Analysis	Docker-Compose environment parity was achieved early, allowing all team members to test the Provider API in an identical runtime.	Task descriptions were initially too broad (e.g., "Audit Security"), making it difficult to track granular progress.	Decomposed large tasks into detailed Child Tasks with specific "Definition of Done" criteria for the API and JWT gap analysis.
Sprint 3: Quality & Closure	High pass rate in BVA testing and successful identification of the "0KB upload" and "Grade range" bugs.	Finalizing the report formatting across four different contributors created stylistic inconsistencies.	Implemented a Final Peer Review phase where one lead editor (Lam) unified the tone and cross-references before submission.

As evidenced by the Jira Timeline and Kanban screenshots, the team maintained a high velocity by addressing "What didn't go well" immediately. The transition from scattered communication to Jira-centric collaboration (as shown in the comments in Section 2.4) allowed for a more rigorous and transparent analysis project.

---

6. Requirements Verification & Risk

6.1 Features Checklist:

Functional requirements define the specific behaviors and services the "Provider" system must offer to its "Consumers." These are derived from the core logic found in the controllers/ and api/ directories of the codebase.

figures 16: Features Checklist

Req. ID	Role	Requirement Description (The System Shall...)	Status
FR-01	User	Authenticate users via the auth.php provider using secure credentials.	✅ Complete
FR-02	Student	Allow the upload of assignment files in specific formats (.pdf, .zip) to the /uploads directory.	✅ Complete
FR-03	Student	Provide a real-time view of assignment status and received grades via the grades.php API.	✅ Complete
FR-04	Instructor	Enable the creation and management of grading rubrics with weighted criteria.	✅ Complete
FR-05	Instructor	Allow for the manual override of automated grades and the addition of qualitative feedback.	✅ Complete
FR-06	System	Send automated JSON confirmation responses upon successful data persistence.	✅ Complete

Non-Functional Requirements (NFR)

Non-functional requirements (or "Quality Attributes") specify the constraints under which the system operates. These ensure the system is robust, secure, and professional.

- NFR-01: Security (Authentication): The system shall transition from session-based state to stateless JWT (JSON Web Tokens) for all API interactions to prevent session hijacking.
- NFR-02: Portability (Containerization): The system shall be fully containerized via Docker, ensuring identical behavior across Windows, Linux, and macOS environments.
- NFR-03: Performance (Latency): The API shall respond to standard GET requests within less than 200ms under a concurrent load of 50 students.
- NFR-04: Data Integrity: The system shall utilize PDO prepared statements for all database interactions to mitigate SQL Injection risks.
- NFR-05: Recoverability: The system shall persist all uploaded files and database records using Docker Volumes to prevent data loss during container restarts.

Use Case Modeling

To visualize these requirements, the following Use Case Diagrams (see Figure X) illustrate the interactions between the primary actors:

1. Student: Primary goal is to submit work and view feedback.
2. Instructor: Primary goal is to manage assignments, rubrics, and the grading engine.

3. System Admin: Primary goal is to manage the Docker infrastructure and database health.

6.2 Risk Management:
A comprehensive risk assessment was conducted to identify technical and project-related threats to the Online Grading System. Each risk is evaluated based on its Probability (P) of occurrence and its Impact (I) on the project's success, using a scale of 1 to 5 (Score = P x 1).

figures 17: Risk Register

Risk ID	Risk Description	Category	P	I	Score	Mitigation Strategy
R-01	Unauthorized API Access	Security	3	5	**15**	Implement stateless JWT authentication and enforce strict CORS policies on the Provider.
R-02	Docker Environment Conflict	Infrastructure	2	4	**8**	Use a single docker-compose.yml source of truth and fixed image versions (e.g., php:8.2-apache).
R-03	SQL Injection Vulnerability	Security	2	5	**10**	Enforce the use of PDO prepared statements for all database interactions within the Model layer.
R-04	Data Loss on Restart	Technical	1	5	**5**	Utilize Docker Volumes to map /uploads and /var/lib/mysql to the host machine for persistence.
R-05	Scope Creep (Analysis)	Project	4	2	**8**	Strict adherence to the WBS and time-boxed sprints managed via the Jira Kanban board.

Probability x Impact Matrix
The identified risks are mapped into the matrix below to prioritize mitigation efforts. Risks scoring above 10 (Red Zone) are treated as critical path items.

- Critical (Score 15-25): R-01 (Security) is the top priority, requiring immediate implementation of robust authentication.
- Moderate (Score 6-12): R-03 and R-05 require regular monitoring during the architectural audit phase.
- Low (Score 1-5): R-04 is considered mitigated by the current Docker configuration but remains on the watch list.

Contingency Planning
In the event of a critical failure (e.g., a database corruption), the team has established a recovery protocol involving the use of the mysql_data volume backups. Furthermore, for the analysis project itself, the use of a remote Git repository (GitHub/GitLab) serves as a contingency against local hardware failure, ensuring the "Analysis Blueprint" remains accessible to all team members at all times.

7. Testing & Quality Assurance (15%)

The Quality Assurance (QA) strategy for the Online Grading System follows a Black-Box Testing methodology, focusing on the system's inputs and outputs without requiring internal code modification. The strategy is designed to verify the "Provider" API's resilience and the "Consumer" UI's usability. Given the project's analytical nature, the focus was placed on Manual Scripted Testing and Boundary Value Analysis (BVA) to identify potential logic gaps in the grading engine.

Test Levels & Scope

The evaluation was conducted across three distinct levels to ensure comprehensive coverage:

Integration Testing: Verifying the communication link between the PHP app container and the MySQL db container. This ensures that data persisted via the API is correctly reflected in the database.

Functional Testing: Validating that every "The system shall..." requirement (defined in Section 6.1) has a corresponding test case.

Security Testing (Vulnerability Audit): Specifically testing the API endpoints for unauthorized access by attempting to reach /api/grades.php without a valid session or token.

Test Environment & Tools

To maintain environment parity, all tests were executed within the Dockerized production-equivalent environment. This ensures that results are not skewed by local OS configurations. Tools used include:

Postman: For auditing RESTful API response codes and JSON payload integrity.

Browser Developer Tools: For monitoring network requests and session cookie behavior.

Manual Test Logs: A structured documentation of pass/fail results for edge-case scenarios.

Quality Gate Criteria

For a feature to be considered "Verified," it must pass the following criteria:

Zero Critical Defects: No security vulnerabilities (e.g., SQL injection) or data loss during container restarts.

Schema Consistency: All API responses must match the documented JSON schema in the API Specification (Section 3.3).

Graceful Failure: The system must provide meaningful error messages to the user when invalid data is submitted.

7.2 Manual Test Log
Following the strategy outlined in Section 7.1, a total of 15 test cases were executed across the functional and security domains. The testing resulted in an 80% Pass Rate, with the remaining 20% identifying "Quality Gaps" that have been logged for future iterations. These results confirm that while the core Provider-Consumer logic is stable, specific edge cases in the grading engine require further refinement.

Key Findings & Defect Analysis
The analysis of the test logs revealed three primary categories of results:

Success - Infrastructure Stability: 100% of tests related to Docker Persistence passed. Database records and file uploads remained intact across container stop and start cycles, validating the effectiveness of the volume mapping strategy defined in Section 4.2.

Success - Core API Logic: The RESTful endpoints for user authentication and assignment retrieval returned the expected HTTP 200/201 status codes with correctly structured JSON payloads. This confirms the reliability of the Model-View-Controller (MVC) decoupling.

Gap - Validation Weakness: Testing revealed that the assignments.php endpoint accepts 0KB files. While the system does not crash (Graceful Failure), it lacks a business-logic check to prevent empty submissions, which has been flagged as a "Low Severity" defect.

Security Audit Results
A significant portion of the analysis focused on the Authentication Gap. While session-based testing passed in a standard browser environment, "Cross-Container" API calls via Postman highlighted the limitations of PHP sessions for stateless scalability. This finding directly supports the recommendation in Section 3.3 to transition to a JSON Web Token (JWT) architecture to ensure more robust security for the Provider API.
Final Quality Assessment
Despite the minor validation gaps, the system meets the Minimum Viable Product (MVP) standards for an academic environment. The high success rate in data integrity and environment parity suggests that the system is ready for a "Beta" release, provided the identified security and validation improvements are addressed in the next sprint.

7.3 Evaluation of Manual Test Plan (Boundary Value Analysis)

The manual test plan was evaluated using Boundary Value Analysis (BVA), a technique focused on testing the "edges" of input ranges where software logic is most likely to fail. For the Online Grading System, this specifically targeted the file upload size limits, grading scales (0–100), and deadline timestamps. By testing these extremes, the team verified the system's input validation and error-handling resilience.

The following table summarizes the "Destructive" test cases used to audit the system's robustness:

figures 18: Destructive Test Case Table

Test ID	Scenario	Input Value	Expected Result	Actual Result	Status
BVA-01	Grade Minimum	-1	System rejects value; returns error.	Value rejected; SQL error handled.	Pass
BVA-02	Grade Maximum	101	System rejects value; returns error.	Value accepted (Validation Gap).	Fail
BVA-03	File Size (Low)	0KB File	System blocks empty submission.	File uploaded successfully.	Fail
BVA-04	File Size (High)	50MB File	Apache/PHP blocks file (exceeds limit).	413 Payload Too Large returned.	Pass
BVA-05	Auth Bypass	Direct URL	Redirect to login or return 401.	Redirected to index.php.	Pass

Analysis of Failures

The failures identified in BVA-02 and BVA-03 represent critical areas for logic refinement:

- The "101 Grade" Issue: The grades.php API lacks a range-check constraint, relying solely on the database column type. This indicates a need for backend validation logic before the Model layer executes the UPDATE query.
- The "0KB Upload" Issue: While the system handles the file physically, it creates a "ghost" submission in the database. A mitigation strategy would be to add a filesize() check in the AssignmentController before persisting metadata.

The evaluation concludes that the Provider API is highly resilient against infrastructure-level failures (e.g., large file crashes) but requires more granular Business Logic Validation. The manual test plan effectively highlighted these gaps, providing a clear roadmap for the "Proposed Target State" of the system.

---

8. Project Closure

8.1 Final Deliverables Checklist:

To ensure full compliance with the Project Portfolio requirements, a final audit of all deliverables was conducted. The following checklist confirms the completion and verification of the system analysis components, as tracked through the "Project Closure" phase of our Jira Kanban board.

figures 20: Final Deliverables Checklist

Deliverable Category	Item Description	Verification Method	Status
Project Management	Work Breakdown Structure (WBS)	Verified against Section 2.1; matches Jira Epics.	Complete

	Project Schedule (Gantt Chart)	Validated via Jira Timeline; reflects 6-week lifecycle.	Complete
	Risk Register & Matrix	Probablity x Impact matrix documented in Section 6.2.	Complete
System Architecture	Provider/Consumer Blueprint	MVC and Docker orchestration mapped in Section 3.1.	Complete
	Data Flow Diagram (Level 1)	Mermaid.js logic verified for submission workflows.	Complete
	API Specification Audit	All RESTful endpoints documented in Section 3.3.	Complete
Software Design	Requirements Catalog	FRs and NFRs labeled and traced in Section 6.1.	Complete
	Use Case Modeling	Role-based interactions (Student/Instructor) visualized.	Complete
Quality Assurance	Manual Test Plan & Results	BVA Destructive testing logged in Section 7.3.	Complete
	Test Strategy Evaluation	Black-box methodology defined in Section 7.1.	Complete
Technical Assets	Source Code Archive	Cleaned and commented code provided in code.docx.	Complete
	Docker Configuration	docker-compose.yml verified for environment parity.	Complete
	Jira Board Screenshot	High-resolution capture of Exercise 3 included.	Complete

The project successfully transitioned through all planned stages of the systems engineering lifecycle. By utilizing a "Stateless Provider" model and a containerized infrastructure, the team has delivered a comprehensive analysis that satisfies both the functional business needs and the non-functional quality attributes (Security, Portability, and Scalability) required for a professional-grade grading system.

*** 8.2 Lessons Learned:**

Reflecting on this project from a Systems Engineering and Project Management perspective has provided several key insights:

What we would do differently:
Systems Engineering: If we started over, we would implement Automated CI/CD pipelines earlier. Manually verifying Docker builds on different Mac architectures (Intel vs. M1/M2) consumed time that could have been saved with an automated build-test pipeline.

Agile Process: Regarding Agile, we would reduce our Sprint length from 2 weeks to 1 week during the "Enhancement Phase." The high complexity of features like JWT and Redis required more frequent "Stand-ups" to address blockers than our 2-week schedule originally allowed.

Requirements Traceability: We learned that maintaining a strict Requirements Traceability Matrix (RTM) is essential when the teacher provides a long list of enhancement ideas, as it prevents "Scope Creep" and ensures no small UI requirement (like Dark Mode) is forgotten during the backend-heavy Sprints.

Appendix

A.1 GitHub Repository Link (Ensure the tutor has access).
GitHub Repository Link: [Insert Team's GitHub Link here]

A.2 Full Meeting Minutes

Meeting 01: Project Initiation & Foundation
Date: February 14, 2026 | Time: 02:00 PM – 03:30 PM
Location: School Library
Attendees: All Members

Agenda:
1. Review CS425 Assignment requirements and "Starter Codebase."
2. Define team roles and skill assessment.
3. Verify local development environment (Docker).

Discussion:
- Team Roles: Agreed that Student A will act as DevOps lead (Docker/Backend). Student B will handle Project Management (Jira/Git). Student C will lead UI/UX (Frontend).
- Infrastructure: Discussion occurred regarding Mac M2 vs Windows WSL2 compatibility. Agreed to use a unified docker-compose setup to avoid "it works on my machine" issues.
- Scope: Decided to prioritize JWT Security and Mobile Responsiveness as our primary differentiators.

Action Items:

- [Member A]: Finalize Docker volume persistence in docker-compose.yml. (Due: 2 weeks)
- [Member B]: Initialize Git repository and setup Jira Kanban board. (Due: 2 weeks)

Meeting 02: Sprint 1 Review (UI/UX) & Sprint 2 Planning
Date: February 28, 2026 | Time: 02:00 PM – 03:00 PM
Location: Online (Zoom)
Attendees: All Members

Agenda:
1. Review Mobile-First Navigation progress.
2. Plan Backend Performance optimizations.

Discussion:
- Frontend: Student C demonstrated the Dark Mode toggle. Agreed to use localStorage so the theme persists on page refresh.
- Performance: Discussion on slow query logs. Agreed to implement SQL B-Tree Indexes on student_id and assignment_id to optimize the grading dashboard.

Action Items:
- [Member C]: Finalize CSS Media Queries for the Hamburger menu. (Due: 2 weeks)
- [Member A]: Draft the new schema.sql with optimized indexes. (Due: 2 weeks)

Meeting 03: Sprint 3 (Security & Features) Mid-Point Sync
Date: March 14, 2026 | Time: 02:00 PM – 03:30 PM
Location: School Cafeteria
Attendees: All Members

Agenda:
1. Secure the Provider App using JWT.
2. Implement Assignment Cloning logic.

Discussion:
- Security: Team decided to move away from PHP Sessions to JWT (JSON Web Tokens) to allow the "External API Consumer" to interact with the system securely.
- Feature: Student B suggested adding "Assignment Cloning" to help instructors save time. Logic will be added to the AssignController.php.

Action Items:
- [Member A]: Implement auth.php middleware for JWT validation. (Due: 2 weeks)
- [Member C]: Build the "Cloning" button and AJAX confirmation modal. (Due: 2 weeks)

Meeting 04: Sprint 4 (QA) & Project Closure

Date: March 28, 2026 | Time: 02:00 PM – 04:00 PM
Location: School Library
Attendees: All Members

Agenda:

1. Final Manual Test Log execution.
2. Review Burndown/Velocity charts for the report.
3. Prepare final ZIP submission.

Discussion:

- QA: Manual testing found a bug in the CSV export where special characters were garbled. Student A fixed the encoding to UTF-8.
- Retrospective: Team noted that Docker troubleshooting was the biggest time-sink in Sprint 2.
- Final Audit: Verified that the RTM (Requirements Traceability Matrix) is 100% complete.

Action Items:

- [Member B]: Compile all sections into the final Portfolio PDF. (Due: 2 weeks)
- [Member A/C]: Generate final SQL dump with Sample Data. (Due: 2 weeks)